Birzeit University
Faculty of Engineering & Technology
Department of Electrical & Computer Engineering

Handwritten Arabic Numerals Recognition (0–9)

Computer Vision Final Project

prepared by:
Sondos Qasarwa 1210259

Instructor: Mr. Aziz Qaroush

March 12, 2026

1 Introduction

Automatic recognition of handwritten digits is an important task in computer vision with applications in document analysis, automated form processing, and digital archiving systems. Accurate recognition enables computers to convert handwritten numerical information into machine-readable form. However, recognizing handwritten Arabic numerals remains challenging because handwriting styles differ significantly between individuals, resulting in variations in shape, thickness, and orientation of digits.

Early approaches to digit recognition commonly relied on handcrafted visual features combined with classical machine learning algorithms such as k-Nearest Neighbors or Support Vector Machines. In recent years, deep learning techniques, particularly Convolutional Neural Networks (CNNs), have demonstrated superior performance by automatically learning hierarchical image representations directly from data [1, 2].

In this project, a system is designed for recognizing handwritten Arabic digits (0–9) using a convolutional neural network–based model. To improve the model's ability to generalize to different handwriting styles, the network incorporates batch normalization and data augmentation during training. Experimental evaluation shows that the proposed model achieves high classification accuracy 97.29% on the dataset, highlighting the effectiveness of CNN-based methods for handwritten Arabic digit recognition.

2 Proposed System

2.1 System Overview

The proposed system performs handwritten Arabic digit recognition using a convolutional neural network (CNN). The input images are first preprocessed by converting them to grayscale and resizing them to 28×28 pixels to ensure a consistent input size for the network. Pixel intensities are normalized to the range $[0, 1]$ to improve numerical stability during training. In addition, An image inversion transformation is also applied to the input images. This preprocessing step is evaluated experimentally and found to improve the recognition performance of the model.

During training, data augmentation is applied to increase the diversity of the training samples and reduce overfitting. The augmented images are then passed through a CNN that extracts hierarchical visual features from the input images and maps them to one of the ten digit classes (0–9). Figure 1 shows the architecture of the proposed CNN model.

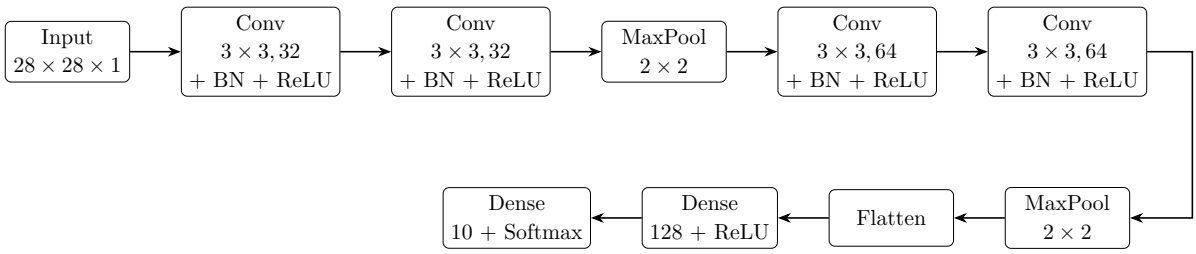

Figure 1: Architecture of the proposed CNN model for handwritten Arabic digit recognition.

2.2 CNN Architecture

The architecture of the proposed network is illustrated in Fig. 1. The model contains two convolutional blocks with 32 and 64 filters respectively. Each block consists of two stacked convolutional layers with 3×3 kernels. Stacking convolution layers allows the network to learn more complex spatial patterns while gradually increasing the receptive field.

Each convolution is followed by batch normalization and a ReLU activation function. Batch normalization stabilizes training and improves convergence, while ReLU introduces non-linearity that enables the network to learn complex feature representations. A 2×2 max-pooling layer is applied at the end of each block to reduce the spatial resolution of the feature maps while preserving the most discriminative features.

After the convolutional feature extraction stage, the feature maps are flattened and passed to a fully connected layer with 128 units. The final output layer consists of 10 neurons with softmax activation, producing the probability distribution over the digit classes.

2.3 Data Augmentation

To improve the generalization ability of the network, several data augmentation techniques are applied during training. These transformations include small rotations (up to 10°), horizontal and vertical shifts (up to 10% of the image size), and zoom transformations (up to 10%). These augmentations simulate natural variations in handwritten digits and help the model learn features that are robust to changes in orientation, position, and scale.

2.4 Training Objective

The network is trained using the sparse cross-entropy loss function, which is suitable for multi-class classification problems with integer class labels. The model parameters are optimized using the Adam optimizer. During training, the objective is to minimize the classification loss so that the predicted probability distribution over the digit classes matches the ground truth labels.

3 Dataset

The experiments are conducted on publicly available Handwritten Arabic Numerals dataset [3], which contains images of handwritten digits from 0 to 9. The dataset consists of 9350 grayscale images distributed across 10 classes, with 935 samples per digit. All images are provided in grayscale format with a spatial resolution of 28×28 pixels.

The dataset is divided into training, validation, and test sets using stratified sampling, with 70% of the samples used for training, 15% for validation, and 15% for testing.

The performance of the proposed system is evaluated using accuracy, precision, recall, and F1-score, computed using macro-averaging across the ten digit classes.

4 Experimental Results

All experimental comparisons are performed on the validation set, and the final selected model is evaluated on the test set.

4.1 CNN Architecture Study

To determine an effective CNN structure for handwritten digit recognition, we compare several architectures that vary in the number of convolution layers per block and the total number of blocks. Each architecture consists of convolutional layers followed by max-pooling, but differs in the depth of feature extraction within each block. The results in Table 1 show that stacking two convolution layers per block improves performance, with the two-block stacked architecture achieving the best validation accuracy. Table 1.

Table 1: **CNN architecture comparison.** Validation accuracy of different convolution block designs.

Architecture	Block	Blocks	Val Acc	Val F1	Params
Single-conv, 2 blocks	Conv $\rightarrow$ Pool	2	91.87%	91.85%	422 K
Single-conv, 3 blocks	Conv $\rightarrow$ Pool	3	94.30%	94.27%	242 K
Double-conv, 3 blocks	Conv$\times$2 $\rightarrow$ Pool	3	95.65%	95.65%	435 K
Double-conv, 2 blocks	**Conv$\times$2 $\rightarrow$ Pool**	**2**	**95.72%**	**95.73%**	**468 K**

4.2 Regularisation Study

To improve generalisation, we evaluate several regularisation techniques using the best architecture from Table 1. In particular, we compare L2 weight decay, dropout with different rates, and batch normalisation. All models are trained without data augmentation to isolate the effect of regularisation. As shown in Table 2, batch normalisation achieves the highest validation accuracy, while dropout provides moderate improvements depending on the drop rate. In contrast, L2 regularisation degrades performance for this task, likely because weight penalties restrict the model's capacity to learn useful features.

Table 2: **Regularisation Ablation.** Effect of different regularisation strategies applied to the best architecture from Table 1.

Regularisation	Val Acc
None (baseline)	95.72%
L2 ($\lambda = 10^{-4}$)	93.16%
Dropout ($p = 0.10$)	93.59%
Dropout ($p = 0.25$)	95.37%
Dropout ($p = 0.40$)	96.29%
Batch Normalisation	**97.01%**

4.3 Data Augmentation Study

After selecting the architecture and regularisation strategy, we examine whether data augmentation further improves generalisation. Two augmentation schemes of different strength are evaluated, and their interaction with batch normalisation and dropout is

analysed. As shown in Table 3, augmentation consistently improves performance, with the combination of mild augmentation (Aug-1) and batch normalisation achieving the best validation accuracy. Stronger augmentation provides no additional benefit.

Table 3: **Data Augmentation Study.**

Augmentation	**BN**	**Dropout**	**Val Acc**
Aug-1 (mild)	–	–	97.29%
Aug-2 (stronger)	–	–	96.58%
Aug-1	–	✓	96.29%
Aug-1	✓	✓	96.86%
Aug-1	✓	–	**98.00%**

Based on these results, the configuration combining the two-block stacked CNN architecture, batch normalization, and data augmentation is selected as the final model for evaluation on the test set.

4.4 Custom CNN vs Alternative Approaches

To show that a custom CNN works well for this task, two alternative approaches are also evaluated in addition to the proposed model: a classical feature-based method (HOG+SVM) and a transfer learning model based on MobileNetV2. Previous work on handwritten digit recognition suggests that custom CNN architectures often perform well on small grayscale datasets, while transfer learning from large natural-image models may suffer from domain mismatch and limited training data. The results on the held-out test set are presented in Table 4. The custom CNN achieves the highest accuracy despite having far fewer parameters than the pre-trained network. While transfer learning remains a promising direction, stronger performance may require larger datasets or additional fine-tuning strategies, which are left for future work.

Table 4: **Final System Comparison on the Test Set.**

Method	**Strategy**	**Test Acc**	**Params**
HOG + SVM (RBF, $C = 10$)	Hand-crafted features	82.89%	–
MobileNetV2† (frozen)	Head only	92.44%	∼2.3 M
MobileNetV2† (fine-tuned)	Partial fine-tune	93.87%	∼2.3 M
Custom CNN (2×Conv + BN + Aug)	**From scratch**	**97.29%**	**468 K**

The proposed custom CNN also achieves Precision = 97.35%, Recall = 97.29%, and F1-score = 97.29%.

5 Discussion

5.1 Error Analysis

Figure 2 presents the confusion matrix of the final CNN model. Most digits are classified correctly, indicating that the network successfully captures the structural patterns of

handwritten digits. However, a small number of misclassifications occur between visually similar digits. The largest confusion is observed between digits 2 and 3, where some instances of digit 2 are predicted as digit 3. This is likely due to similarities in their handwritten stroke patterns and variations in writing styles. Overall, the errors are sparse and concentrated among visually similar classes.

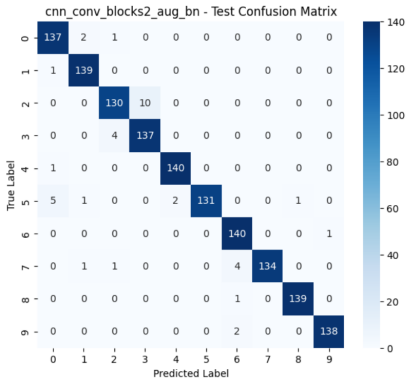

Figure 2: Confusion matrix of the proposed CNN model on the test set.

5.2 Strengths, Limitations, and Conclusion

The proposed CNN architecture achieves strong recognition performance, reaching 97.29% accuracy on the test set while maintaining a relatively compact model size. This result demonstrates that task-specific CNN architectures can be highly effective for handwritten digit recognition, particularly when the available dataset is relatively small and domain-specific.

The strong performance of the model can be attributed to several design choices explored in the experiments. Stacked convolutional layers allow the network to capture increasingly complex stroke patterns, while batch normalization stabilizes training and improves generalization. Data augmentation further improves robustness by exposing the model to variations in digit orientation and position, helping the network learn more invariant feature representations.

Despite these strengths, some limitations remain. A small number of errors occur between visually similar digits, particularly digits 2 and 3, which share similar stroke structures in certain handwriting styles. In addition, the dataset size is relatively limited compared to large-scale vision datasets, which may restrict the diversity of handwriting variations observed during training.

Future work may explore larger datasets, and improved transfer-learning techniques to further enhance performance. Overall, the results demonstrate that a carefully designed lightweight CNN can provide an accurate and efficient solution for handwritten Arabic digit recognition.

References

[1] F. Siddique, S. Sakib, and M. A. B. Siddique, "Recognition of handwritten digit using convolutional neural network in python with tensorflow and comparison of performance for various hidden layers," in *2019 5th International Conference on Advances in Electrical Engineering (ICAEE)*, 2019, pp. 541–546.

[2] A. Ashiquzzaman and A. K. Tushar, "Handwritten arabic numeral recognition using deep learning neural networks," 2017. [Online]. Available: https://arxiv.org/abs/1702.04663

[3] H. Azis and I. N. Saly, "Handwritten arabic numerals (0–9) image dataset," 2024. [Online]. Available: https://doi.org/10.17632/5hpkf8v7bg.1